Tecnologias de Comunicação para Sistemas Críticos
Laboratory Class PL4 2026/27 Edition – 3 hours
Arduino & Wireless networks (Wi-Fi & Bluetooth)
(Some exercises adapted from AMBIA)

Table of Contents

1. Wi-Fi....1
RSSI signal strength and meaning....2
1.1 [Lab04.ex01] Use the Arduino to scan for the available Wi-Fi networks....3
Questions....5
1.2 [Lab04.ex02] Set Arduino as an AP....5
Questions....7
1.3 [Lab04.ex03] Show a log of the last 10 LED on/off actions available in the serial monitor....8
1.4 [Lab04.ex04] multi-user concurrent handling....9
2. Bluetooth....13
2.1 [Lab04.ex05] Bluetooth Example....13
Questions....16
3. [Lab04.ex06] Challenge....17

Objectives for the class:

- Understand Wi-Fi and Bluetooth wireless communication protocols
- Use Arduino as a platform for practical experiments
- Execute lab exercises to scan Wi-Fi networks and set up as AP
- Establish Bluetooth communication between Arduino and mobile devices
- Combine Wi-Fi, Bluetooth, and TCP/IP in an integrated challenge

1. Wi-Fi

Wi-Fi uses multiple parts of the IEEE 802 protocol family and is designed to interwork seamlessly with its wired sibling, Ethernet. Compatible devices can network through wireless access points to each other as well as to wired devices and the Internet. The different versions of Wi-Fi are specified by various IEEE 802.11 protocol standards, with the different radio technologies determining radio bands,

and the maximum ranges, and speeds that may be achieved. Wi-Fi most commonly uses the 2.4 gigahertz (120 mm) UHF and 5 gigahertz (60 mm) SHF radio bands; these bands are subdivided into multiple channels.

Arduinos with a Wi-Fi component can be used for different purposes.

RSSI signal strength and meaning

Received Signal Strength Indicator

Signal Strength	TL;DR		Required for
-30 dBm	Amazing	Max achievable signal strength. The client can only be a few feet from the AP to achieve this. Not typical or desirable in the real world.	N/A
-67 dBm	Very Good	Minimum signal strength for applications that require very reliable, timely delivery of data packets.	VoIP/VoWi-Fi, streaming video
-70 dBm	Okay	Minimum signal strength for reliable packet delivery.	Email, web
-80 dBm	Not Good	Minimum signal strength for basic connectivity. Packet delivery may be unreliable.	N/A
-90 dBm	Unusable	Approaching or drowning in the noise floor. Any functionality is highly unlikely.	N/A

Key points:

- **dBm** is a unit for signal strength. The closer to 0 (less negative), the stronger the signal.
- **-30 dBm:** Almost never seen in typical environments; very close to the AP. Not usually desirable, as it may indicate the client is using unneeded power.
- **-67 dBm:** Recommended minimum for reliable, time-sensitive uses, like voice or video calls.
- **-70 dBm:** Sufficient for things like web browsing and checking emails.
- **-80 dBm:** You might be able to connect, but expect issues with reliability—dropped connections, delays.
- **-90 dBm:** The signal is so weak that devices likely cannot sustain a connection at all.

In short: Higher (less negative) dBm values mean stronger and more reliable Wi-Fi signals, suitable for more demanding applications. Lower values lead to unreliable or unusable connectivity.

The following two examples show how an Arduino can be used to **scan the area for available networks (Lab04.ex01)** and how it can be used as an **Access Point (AP) (Lab04.02)**.

1.1 [Lab04.ex01] Use the Arduino to scan for the available Wi-Fi networks

Scanning Wi-Fi Networks

- Use Arduino to detect available Wi-Fi networks
- Learn about IEEE 802.11 standards
- Evaluate signal strengths: From Excellent (-30 dBm) to Unusable (-90 dBm)

```
#include "WiFi.h"
void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);  // Set Wi-Fi to station mode
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();
  delay(100);
  Serial.println("Setup done");
}

void loop() {
  // put your main code here, to run repeatedly:
  Serial.println("scan start");
  int n = WiFi.scanNetworks();  // Returns the number of networks found
  Serial.println("scan done");
  if (n == 0) {
    Serial.println("no networks found");
  } else {
    Serial.print(n);
    Serial.println(" networks found");
    for (int i = 0; i < n; i++) {  // Print SSID and RSSI for each network
found
      Serial.print(i + 1);
      Serial.print(": ");
      Serial.print(WiFi.SSID(i));
```

```
      Serial.print(" (");
      Serial.print(WiFi.RSSI(i)); //Gets the signal strength of the
connection to the router
      Serial.print(")");
      Serial.println((WiFi.encryptionType(i) == WIFI_AUTH_OPEN) ? " " :
"*");
      delay(10);
    }
  }
  Serial.println("");
  delay(10000);
}
```

The WiFi.h library provides all the functions needed to work with Wi-Fi on boards like the ESP32.

The code of Arduino is divided in two main functions: *setup* and *loop*. The *setup* function is first executed every time the Arduino is powered or every time a reset is performed.

- **Serial.begin(9600):** Starts serial communication so you can see output in the Serial Monitor.
- **WiFi.mode(WIFI_STA):** Sets the board to "station" mode, meaning it will look for Wi-Fi networks instead of acting as a hotspot.
- **WiFi.disconnect():** Makes sure the board isn't connected to any network before scanning.
- **delay(100):** Waits 100 milliseconds to ensure the Wi-Fi hardware is ready.
- **Serial.println("Setup done"):** Prints a message to confirm setup is complete.

Next, the *loop* function will be executed in an infinite loop.

- **Serial.println("scan start"):** Announces the start of a Wi-Fi scan.
- **WiFi.scanNetworks():** Scans for nearby Wi-Fi networks and returns how many it found.
- **If no networks:** Prints "no networks found".
- **If networks found:**
    - Prints the number of networks.
    - Loops through each network and prints:

        - The network number (starting at 1)
        - The SSID (network name)
        - The RSSI (signal strength, in dBm)
        - Whether the network is open (" ") or secured ("*")
- **delay(10000):** Waits 10 seconds before scanning again, so you don't flood the serial monitor

Questions

1. What does WiFi.mode(WIFI_STA), and why is it important for scanning?
2. How can you tell if a network is open or secured from the output?
3. What does a more negative RSSI value (like -80 dBm) mean about the signal?

Put here the answer:

1.2 [Lab04.ex02] Set Arduino as an AP

The next example uses an Arduino as Access Point (AP)

- Configure Arduino to act as a Wi-Fi AP
- Control an LED via web server hosted on Arduino

```
#include <WiFi.h>
#include <WiFiClient.h>
#include <WiFiAP.h>

#define LED_BUILTIN 2  // Set the GPIO pin where you connected your test LED or
comment this line out if your dev board has a built-in LED
const char *ssid = "comcs2627m1agXX"; // Change XX to your group's ID
const char *password = "12345678";  // remove if you want the AP to be open.

WiFiServer server(80);

void setup() {
  delay(5000);
  pinMode(LED_BUILTIN, OUTPUT);
  Serial.begin(9600);
  Serial.println();
  Serial.println("Configuring access point...");
  WiFi.softAP(ssid, password);
  IPAddress myIP = WiFi.softAPIP();
  Serial.print("AP IP address: ");
  Serial.println(myIP);
  server.begin();
  Serial.println("Server started");
```

```
}

void loop() {
  String currentLine;
  WiFiClient client = server.available();  // listen for incoming clients
  if (client) {                           // if you get a client,
    while (client.connected()) {          // loop while the client's connected
      if (client.available()) {           // if there's bytes to read from the
client,
        char c = client.read();           // read a byte, then
        if (c == '\n') {                  // if the byte is a newline character
          // if the current line is blank, that's the end of the client HTTP
request, so send a response:
          if (currentLine.length() == 0) {
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type:text/html");
            client.println();  // the content of the HTTP response follows the
header:
            client.print("Click <a href=\"/H\">here</a> to turn ON the LED.<br>");
            client.print("Click <a href=\"/L\">here</a> to turn OFF the LED.<br>");
            client.println();
            break;
          } else {
            currentLine = "";
          }
        } else if (c != '\r') {  // if you got anything else but a carriage return
character,
          currentLine += c;      // add it to the end of the currentLine
        }
        // Check to see if the client request was "GET /H" or "GET /L":
        if (currentLine.endsWith("GET /H")) {
          digitalWrite(LED_BUILTIN, HIGH);  // GET /H turns the LED on
        }
        if (currentLine.endsWith("GET /L")) {
          digitalWrite(LED_BUILTIN, LOW);  // GET /L turns the LED off
        }
      }
    }  // close the connection:
    client.stop();
  }
}
```

This code will turn the Arduino into an AP and provides a server that controls, in this case, the Arduino's led.

The libraries allows the ESP32 to act as a Wi-Fi access point, accept clients, and establish network server functionality.

- LED_BUILTIN sets which pin is used for the LED you'll control.
- ssid and password configure the access point users will connect to (change "99" for your own group).
- WiFiServer server(80); creates a web server on port 80 (default for HTTP).

The Setup:

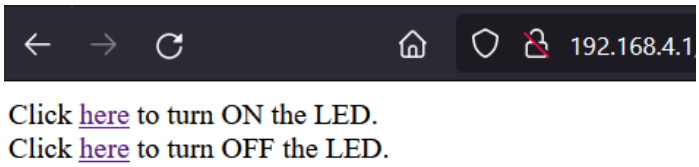
- Configures the board as a Wi-Fi access point with the specified SSID and password.
- Shows the IP address for connecting from your phone/computer.
- Starts the HTTP web server.

The Loop:

- The board waits for a client (usually your phone/computer via Wi-Fi) to connect.
- When a user opens the AP's IP address in a browser, the server receives an HTTP request.
- It processes the request: when a user clicks "here" to turn ON or OFF, the browser sends /H or /L requests to the server.
- When it sees "GET /H", the board's LED is turned ON; "GET /L" turns it OFF.
- The webpage refreshes each time, allowing you to toggle the LED directly from your device.
- After the HTTP response, the client is disconnected until the next request.

Verify the IP using the Arduino monitor.

Using your laptops or mobile phones select the network "`comcs2627m1agXX`"(verify the id of your AP), connecting your newly created AP. Try access the created web page to control the LED using the IP of the Arduino:



Questions

- If more than one user connects at the same time, how would the ESP32 handle the web requests?

- How could you modify the web interface to display the current state of the LED?
- Suggest changes to the program so that a log of the last 10 LED on/off actions is available in the serial monitor.

Put here the answer:

1.3 [Lab04.ex03] Show a log of the last 10 LED on/off actions available in the serial monitor

To log the last 10 LED on/off actions and display them in the **Serial Monitor**, You'll need to:

- Create a **circular buffer** (array) to store the log.
- Maintain an **index** that wraps around after reaching 10.
- Print the entire log to the Serial Monitor whenever the LED is toggled.

Change your code accordingly.

```
const int LOG_SIZE = 10;

String ledLog[LOG_SIZE];

int logIndex = 0;


void logLedAction(String action) {

  // Save action to the log

  ledLog[logIndex] = action;

  logIndex = (logIndex + 1) % LOG_SIZE;


  // Print the log to Serial Monitor

  Serial.println("[LED Action Log]");
```

```
  for (int i = 0; i < LOG_SIZE; i++) {
    int idx = (logIndex + i) % LOG_SIZE;
    if (ledLog[idx].length() > 0) {
      Serial.println(ledLog[idx]);
    }
  }
}
…
logLedAction("LED ON at " + String(millis()) + " ms");
…
logLedAction("LED OFF at " + String(millis()) + " ms");
…
```

This will let you see the **last 10 LED ON/OFF actions with timestamps** in the Serial Monitor each time the LED state changes.

1.4 [Lab04.ex04] multi-user concurrent handling

If more than one user connects at the same time, the ESP32 (using your current code and the **WiFiServer** class) handles web requests **one at a time** and **sequentially**, not in parallel.

What happens under the hood?

- The line WiFiClient client = server.available(); checks if there's a new incoming client.
- If a client connects, the server starts interacting **only with that client** inside the while (client.connected()) loop.
- **Other clients trying to connect at the same time** must wait until:
    - The current client's session is finished (client.stop(); is called).
    - The ESP32 moves on to the next server.available().

Implications:

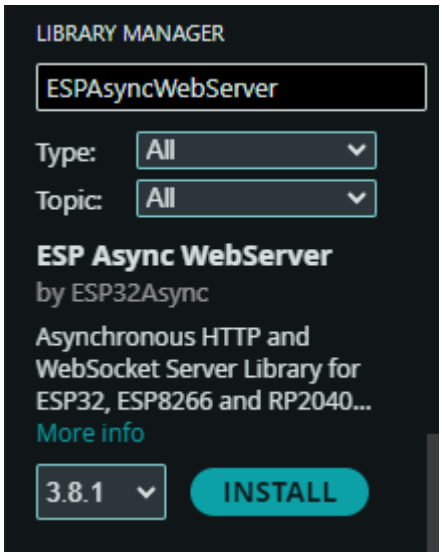
- **Single connection at a time:** Only one client is actively served; others must wait their turn.

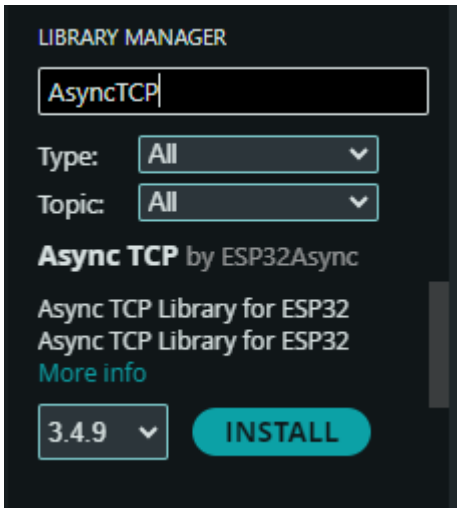
- **Requests are queued:** Additional incoming clients queue up or may be dropped, depending on timing and WiFi stack settings.
- **Simple server logic:** This is typical behavior for simple microcontroller servers with limited resources.

If you need to handle multiple clients (concurrency):

- You would need to adapt your code to **accept and process multiple clients**, such as:
    - Using asynchronous libraries like **ESPAsyncWebServer** (which allows handling multiple connections at once).
    - Managing a pool of WiFiClient objects in your loop and servicing them as needed

To run the next code, two additional libraries are required (ESPAsyncWebServer and AsyncTCP), In Arduino IDE, go to: Sketch > Include Library > Manage Libraries...:





handle multiple clients:

```
#include <WiFi.h>
#include <ESPAsyncWebServer.h>  // Include the AsyncWebServer library

#define LED_BUILTIN 2  // Set the GPIO pin where you connected your test
LED or comment this line out if your dev board has a built-in LED
const char *ssid = "comcs2627m1agXX"; // Change XX to your group's ID
const char *password = "12345678";  // remove if you want the AP to be
open.
```

```
AsyncWebServer server(80);

const int LOG_SIZE = 10;
String ledLog[LOG_SIZE];
int logIndex = 0;

void logLedAction(String action) {
  // Save action to the log
  ledLog[logIndex] = action;
  logIndex = (logIndex + 1) % LOG_SIZE;

  // Print the log to Serial Monitor
  Serial.println("[LED Action Log]");
  for (int i = 0; i < LOG_SIZE; i++) {
    int idx = (logIndex + i) % LOG_SIZE;
    if (ledLog[idx].length() > 0) {
      Serial.println(ledLog[idx]);
    }
  }
}


void setup() {
  delay(5000);
  pinMode(LED_BUILTIN, OUTPUT);
  Serial.begin(9600);
  Serial.println("[Lab04.ex04] AP with mitiple clients");
  Serial.println();
  Serial.println("Configuring access point...");
  WiFi.softAP(ssid, password);
  IPAddress myIP = WiFi.softAPIP();
  Serial.print("AP IP address: ");
  Serial.println(myIP);

  // Serve the main page
  server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
    String html = "<h1>ESP32 LED Control</h1>\
      <button onclick=\"location.href='/on'\">Turn ON LED</button>\
      <button onclick=\"location.href='/off'\">Turn OFF LED</button>";
    request->send(200, "text/html", html);
  });

  // Turn LED on
  server.on("/on", HTTP_GET, [](AsyncWebServerRequest *request){
    digitalWrite(LED_BUILTIN, HIGH);
```

```
    logLedAction("LED ON at " + String(millis()) + " ms");
    request->redirect("/"); // Redirect back to main page
  });

  // Turn LED off
  server.on("/off", HTTP_GET, [](AsyncWebServerRequest *request){
    digitalWrite(LED_BUILTIN, LOW);
    logLedAction("LED Off at " + String(millis()) + " ms");
    request->redirect("/"); // Redirect back to main page
  });

  server.begin();
  Serial.println("Server started");
}

void loop() {
  // No need to poll clients!
}
```

This solution starts to create an asynchronous web server on port 80 (standard HTTP).

Setup:

- **Serial.begin(9600);**: Starts serial communication for debugging.
- **pinMode(LED_BUILTIN, OUTPUT);**: Sets up the LED pin as output.
- **WiFi.softAP(ssid, password);**: Starts the ESP32 as a Wi-Fi Access Point with the given credentials.
- **Display AP's IP** in the Serial Monitor.

The system then **Set up Web Server Routes**

Main Page ("/")

- When a user visits the root URL (/), the server sends an HTML page with two buttons ("Turn ON LED" and "Turn OFF LED").

Turn LED ON

- When /on is accessed (e.g. clicking the button), the LED turns ON and the user is redirected back to the main page.

Turn LED OFF

- When /off is accessed, the LED turns OFF and the user is redirected to the main page.

Loop:

- No code needed here because **ESPAsyncWebServer handles everything in the background, asynchronously**.

2. Bluetooth

Bluetooth technology allows devices to communicate with each other without cables or wires. Bluetooth relies on short-range radio frequency, and any device that incorporates the technology can communicate as long as it is within the required distance. The technology is often used to allow two different types of devices to communicate with each other. It is an electronics "standard," which means that manufacturers that want to include this feature have to incorporate specific requirements into their electronic devices. These specifications ensure that the devices can recognize and interact with other devices that use Bluetooth technology.

2.1 [Lab04.ex05] Bluetooth Example

The next ESP32 Arduino sketch lets you control an LED using Bluetooth, such as from the Serial Bluetooth Terminal app on your phone.

- **Bluetooth Communication**
    - Use Serial Bluetooth Terminal app to control Arduino LED via Bluetooth
    - ESP32 setup and communication logic
    - Comparison of Android device/emulator use

For the bluetooth example you will have to use your mobile device's Bluetooth (Android only) or use an Android emulator to connect to Arduino's Bluetooth. Download the "Serial Bluetooth Terminal" from the Play Store, install it to control the Arduino led using Bluetooth.

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;
```

```
#define LED_BUILTIN 2  // Set the GPIO pin where you connected your test LED
or comment this line out if your dev board has a built-in LED

char Incoming_value = 0;  //Variable for storing Incoming_value
void setup() {
  delay(5000);
  pinMode(LED_BUILTIN, OUTPUT);

  Serial.begin(9600);


  /* If no name is given, default 'ESP32' is applied */
  /* If you want to give your own name to ESP32 Bluetooth device, then */
  /* specify the name as an argument SerialBT.begin("comcs2324m1ag99");  */
  if(!SerialBT.begin("comcs2627m1agXX")){  //change XX to your group´s id
    Serial.println("An error occurred initializing Bluetooth");
  }else{
    Serial.println("Bluetooth initialized");
  }

  Serial.println("Bluetooth Started! Ready to pair...");  //Sets the data
rate in bits per second (baud) for serial data transmission

}

void loop() {
  if (SerialBT.available()) {

    Incoming_value = SerialBT.read();
    Serial.println(Incoming_value);  //Print Value of Incoming_value in
Serial monitor

    if (Incoming_value == '1'){       //Checks whether value of
Incoming_value is equal to 1
      digitalWrite(LED_BUILTIN, HIGH);         //If value is 1 then LED
turns ON
      SerialBT.println("LED turns ON");
    } else if (Incoming_value == '0') { //Checks whether value of
Incoming_value is equal to 0
      digitalWrite(LED_BUILTIN, LOW);          //If value is 0 then LED
turns OFF
      SerialBT.println("LED turns OFF");
    }

  }
```

```
}
```

- **BluetoothSerial.h**: Adds Bluetooth SPP (Serial Port Profile) support — lets the ESP32 communicate wirelessly as a Bluetooth device.
- **SerialBT**: A Bluetooth serial object; initialize it to start the Bluetooth service.
- **LED_BUILTIN**: Choose the LED pin (often GPIO2 on ESP32).
- **Incoming_value**: Stores each incoming character received over Bluetooth.

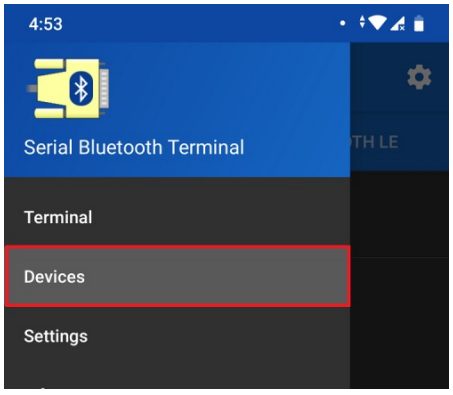
The Setup:

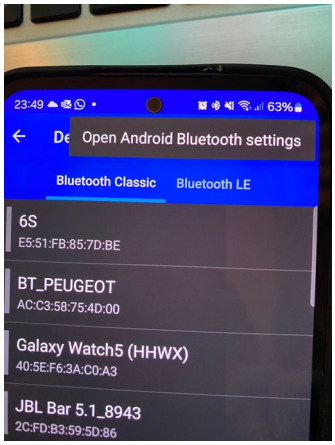
- Waits 5 seconds (gives some boards time to initialize).
- Sets the LED pin as an output, so you can control it.
- Starts serial monitor (for debug output).
- Launches Bluetooth with a custom device name (change "99" to your group´s ID). If initialization fails, prints an error. Otherwise, announces readiness.

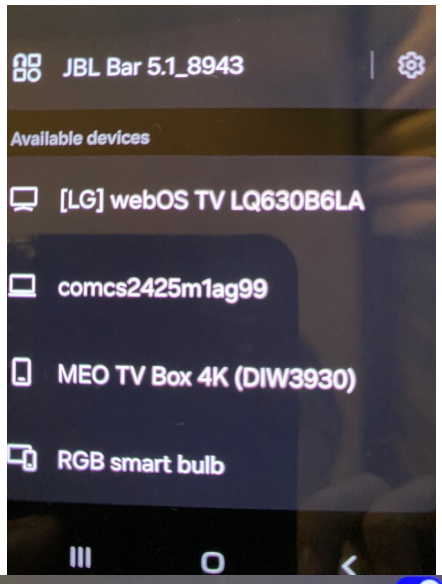
The Loop:

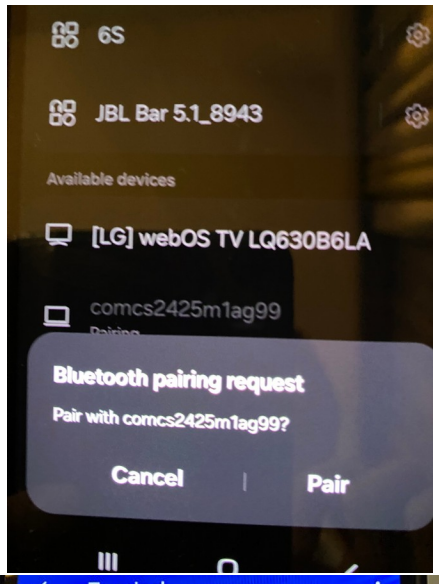
- Checks if there's incoming data over Bluetooth.
- Reads a character; prints it to serial (for monitoring).
- If the character is '1', turn the LED ON and reply "LED turns ON" over Bluetooth.
- If '0', turn the LED OFF and reply accordingly.

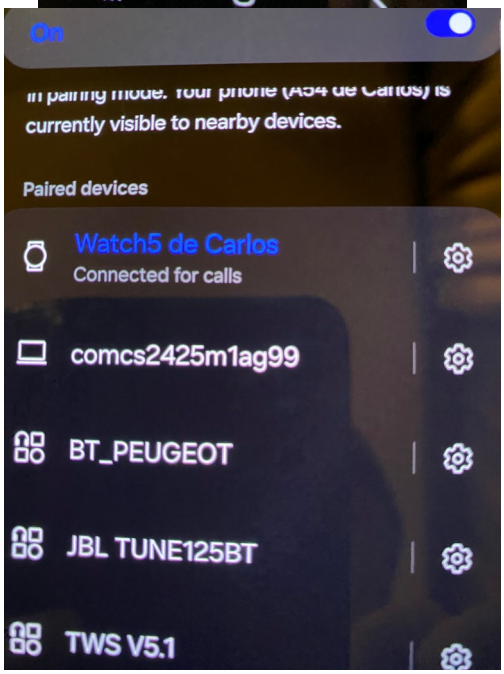
Using **Serial Bluetooth Terminal** android app to connect to the ESP32 (Devices).

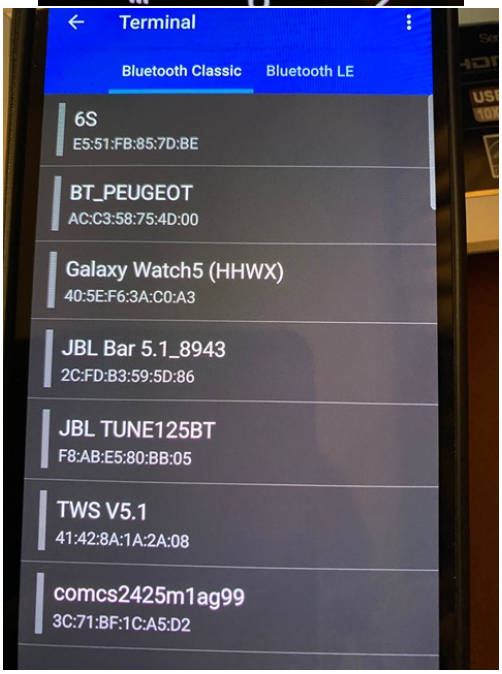


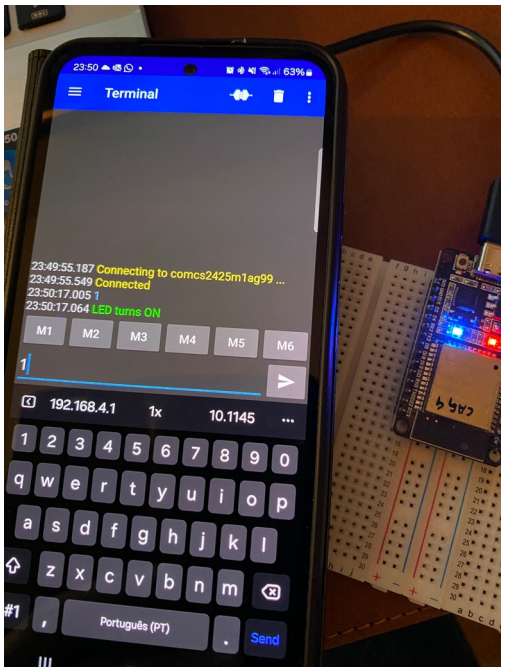


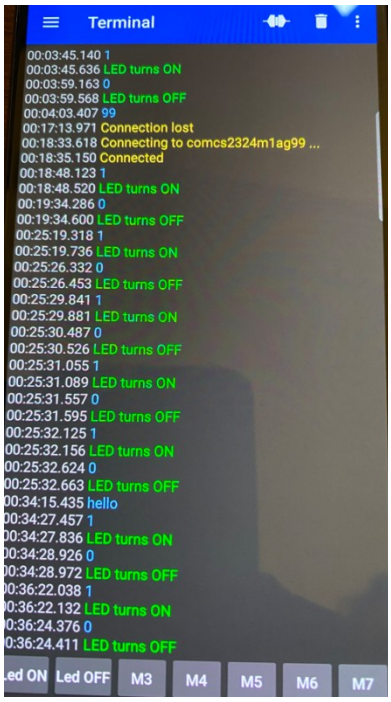

JBL Bar 5.1_8943
Available devices
[LG] webOS TV LQ630B6LA
comcs2425m1ag99
MEO TV Box 4K (DIW3930)
RGB smart bulb


6S
JBL Bar 5.1_8943
Available devices
[LG] webOS TV LQ630B6LA
comcs2425m1ag99
Bluetooth pairing request
Pair with comcs2425m1ag99?
Cancel
Pair


On
currently visible to nearby devices.
Paired devices
Watch5 de Carlos
Connected for calls
comcs2425m1ag99
BT_PEUGEOT
JBL TUNE125BT
TWS V5.1


Terminal
Bluetooth Classic
Bluetooth LE
6S
E5:51:FB:85:7D:BE
BT_PEUGEOT
AC:C3:58:75:4D:00
Galaxy Watch5 (HHWX)
40:5E:F6:3A:C0:A3
JBL Bar 5.1_8943
2C:FD:B3:59:5D:86
JBL TUNE125BT
F8:AB:E5:80:BB:05
TWS V5.1
41:42:8A:1A:2A:08
comcs2425m1ag99
3C:71:BF:1C:A5:D2





Questions

1. Why do you need to start Bluetooth with SerialBT.begin("...")?
2. What feedback does the ESP32 give after each LED action?
3. What would happen if you sent other characters (not '1' or '0')?

Put here the answer:

3. [Lab04.ex06] Challenge

For today's challenge it is time to combine the previous examples along with what you have learned about TCP. You have to install the Serial Bluetooth Terminal from the Play Store.

Combine Wi-Fi, Bluetooth, and TCP/IP communication:

- Setup:

- TCP Server (PC)
- Arduino as client
- Mobile device running Serial Bluetooth Terminal

- Use cases:
  - Turn Arduino LED on/off via mobile, notify server of status
  - Implement user, server address, uptime, and custom commands

Using the TCP server from previous classes, the Arduino as client and a mobile device (Android), create the following setup.

TCP Server <-> Arduino <-> Mobile Device (Serial Bluetooth Terminal)



The user will send messages through the mobile device. If the message is “on/off” it will turn on/off the Arduino’s led and notify the server (the server only needs to print the message) with “The led has been turned on/off” (according to its current status), and the Mobile Device should also print the same message. Other messages that the user can send:

1. User – current user accessing the server
2. Address – the server’s ip address and port
3. Uptime – returns the pc uptime (for linux/mac you can use *uptime* to get this information)
4. <other> - other commands that you would like to add to this list

Put your code on a repository (**comcs2627m1ag**<ID>) using the following structure:

Lab04.ex06

TCPServer

Arduino

Server should have the C code, and the Arduino should have the Arduino sketch code.

Create a readme file indicating the user’s messages implemented.

Notes on: https://moodle.isep.ipp.pt/mod/resource/view.php?id=182479

Useful Git commands:

```
git add -A
git commit -a -m “Lab04.ex03”
git pull
git push
```